



基於YOLO之 PCB 缺孔 (Missing Hole) 瑕疵檢測系統

411287031 蕭宇辰

411287051 羅詠倫

檢測物件: 缺孔 (Missing Hole) 特徵分析

形狀

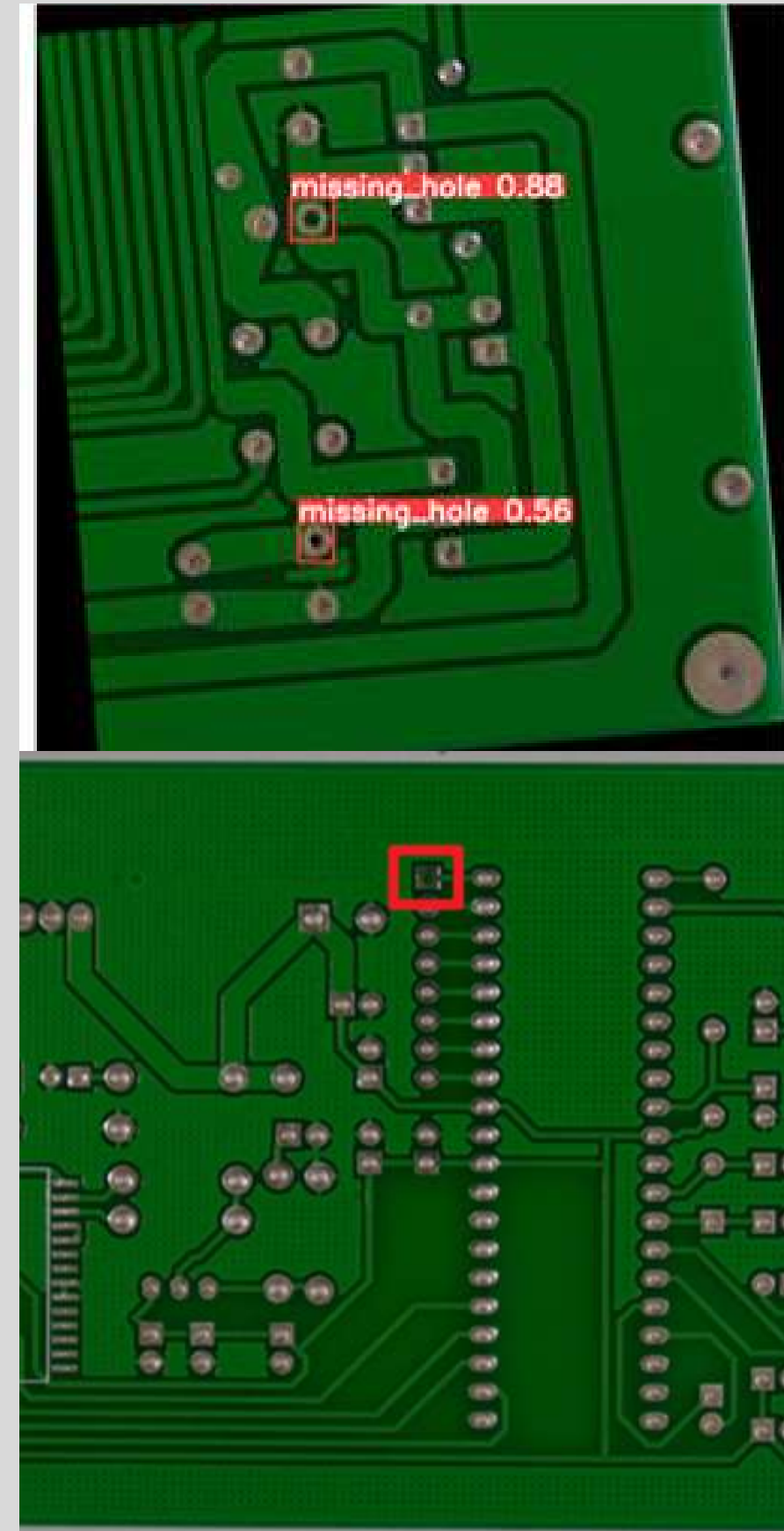
原本應有實體焊錫覆蓋，瑕疵處則呈現中空圓形缺口，邊緣略不規則。

顏色

正常會有焊錫的反光（如銀白色），缺孔處因沒有錫而呈現深暗色，與周圍銅環的明顯反光形成強烈對比。

紋理

缺少正常焊錫的立體或環形紋理，內部呈現中空狀態，表面平滑或直接露出底層基板。



檢測方法－硬體規格

FOV :

- 檢測對象：一般PCB，約為50×100 mm
- ROI 擷取：拍攝後僅擷取畫面正中央的感興趣區域 (ROI) 分析。
- 固定拍攝距離：將手機固定於距離待測 PCB 表面約 10 至 15 公分處，確保每一次拍攝標準板與待測板時，光學放大倍率絕對一致。

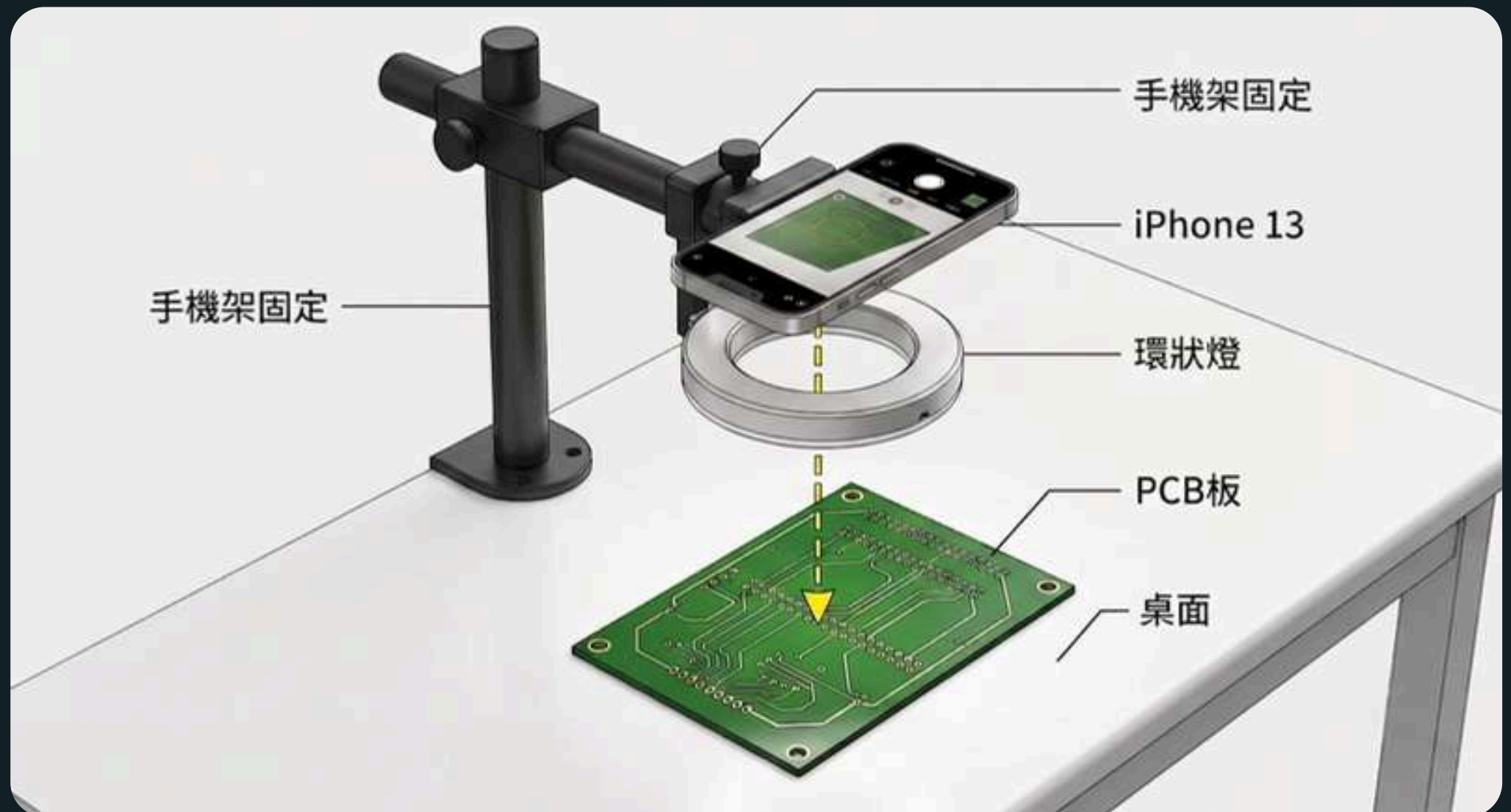
鏡頭 &
相機 & :
打光

- 相機 (iPhone 13 主鏡頭)：1200萬畫素，單像素精度可達 0.05 mm/pixel，足以精確解析微小缺孔的中空邊界。
- 鏡頭設定 (內建廣角 f/1.6)：統一使用 1x 焦段進行純光學成像。拍攝時強制開啟「鎖定對焦與曝光 (AE/AF Lock)」，避免手機自動測光導致前後影像亮度不均。
- 打光 (LED 均勻光源)：使用外部環形燈均勻白光照射，能使未鑽穿的實體表面反光，而真正的缺孔呈現深邃暗色。



檢測方法－機構設計

透過懸臂支架與夾具固定，將 iPhone 13 與緊貼鏡頭下方的環狀光源穩固鎖定於待測 PCB 正上方。能抑制周邊環境或弱用手拿的晃動，防範因鏡頭晃動引起的動態模糊與對焦平面飄移。

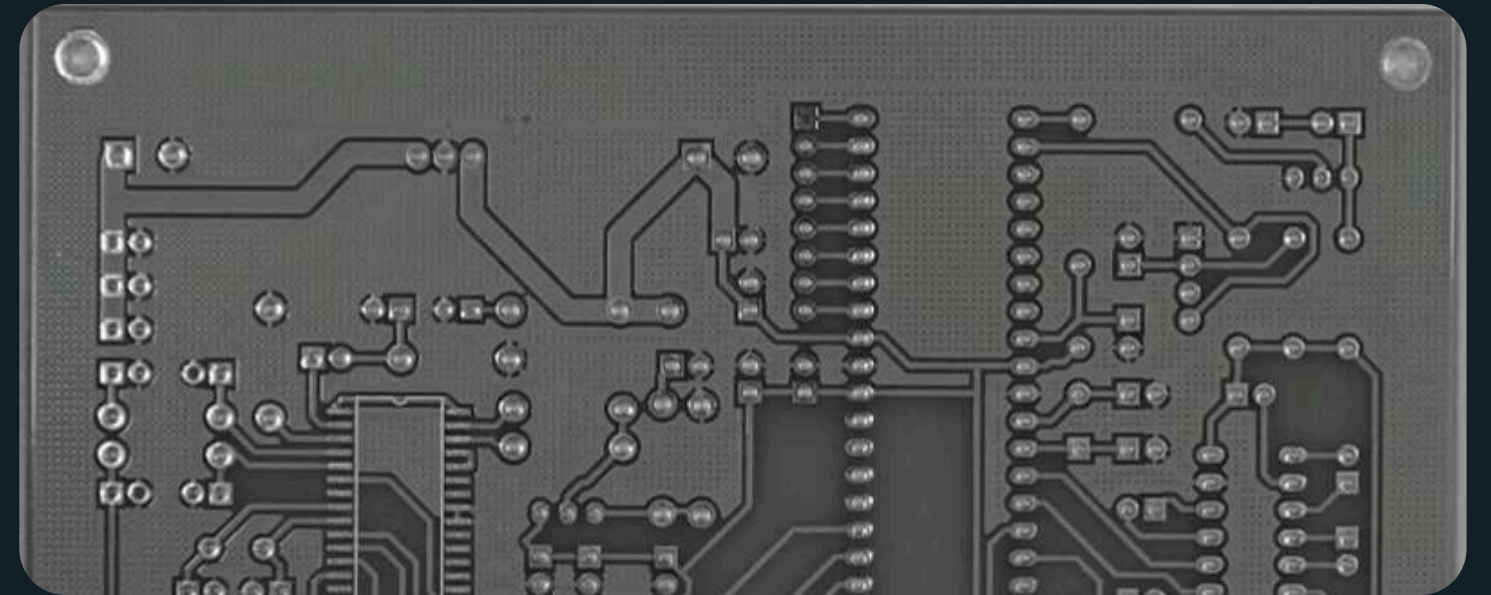


檢測方法－影像處理技術說明

1. 影像灰階化 (Grayscale Conversion)

概念：剔除與孔洞幾何形狀無關的色彩資訊，將三通道的彩色影像降維成單通道的明暗影像。

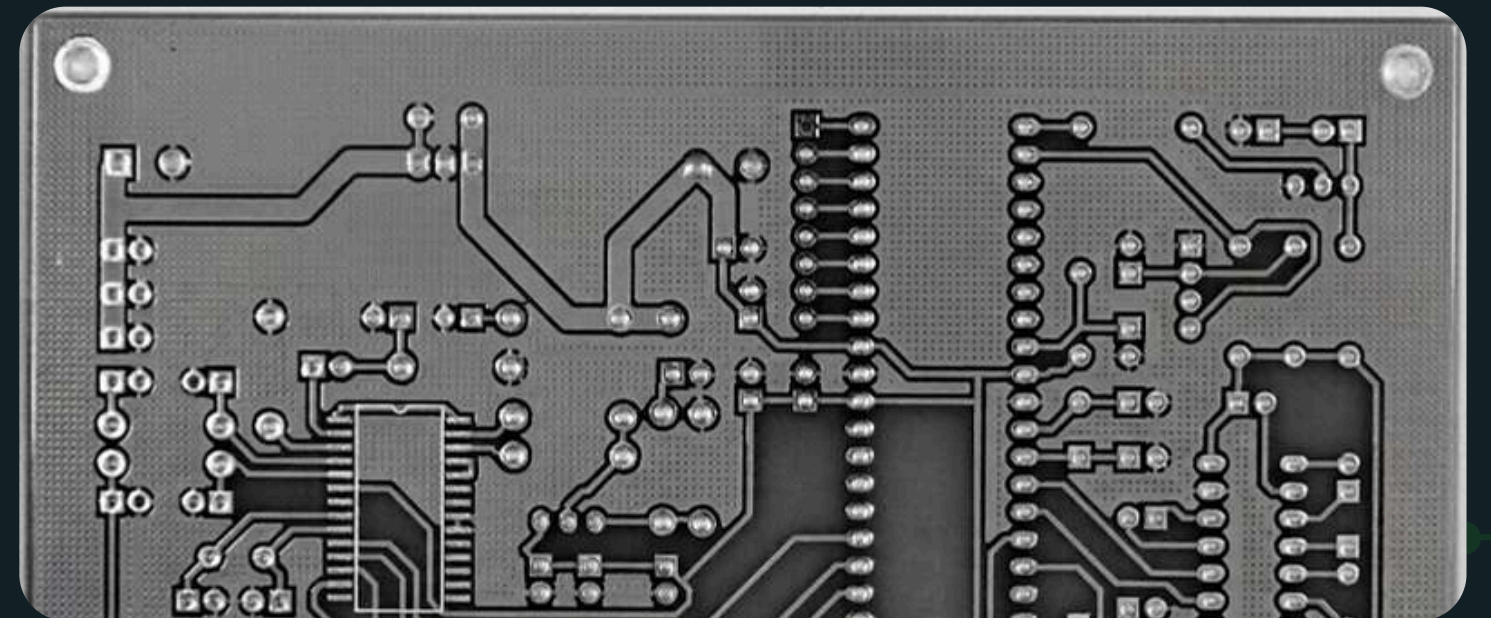
主要目的：大幅減少資料運算量以提升檢測速度，並純化「實體反光」與「中空暗部」的灰階對比特徵。



2. 自適應直方圖均衡化 (CLAHE)

概念：對影像局部區域進行動態的對比度調整，並限制雜訊放大。

主要目的：修正行動裝置拍攝時容易產生的局部光照不均或邊角陰影，同時銳化通孔與邊界的反差，協助 YOLO 模型更精準地定位與辨識瑕疵。



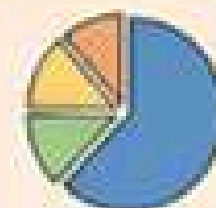
檢測流程

[階段一：開源資料集準備與格式化 (模型建立基礎)]

1. 座標正規化：解析 XML 標註框，絕對座標轉 YOLO 歸一化相對座標

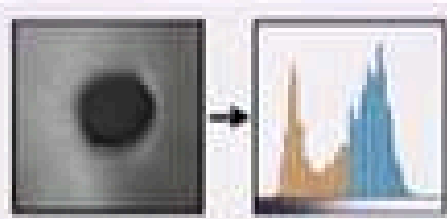


2. 科學化切分：自動建構 YOLO 目錄結構，資料集隨機洗牌，切分 80% 訓練 / 10% 驗證 / 10% 測試

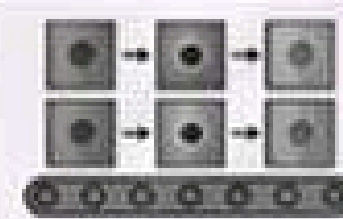


[階段二：核心影像前處理管線 (特徵強化管線)]

1. 參數原型驗證：單張影像測試，確立最佳 CLAHE (自適應直方圖均衡化) 強化策略



2. 批次自動化：將『灰階化 + CLAHE』管線套用至全資料集，產出對比度一致的增強版影像



[階段三：YOLO 瑕疵偵測模型訓練與評估 (深度學習)]

1. 網路迭代訓練：載入 YOLOv8n 架構，輸入固定尺寸影像，迭代訓練，權重收斂



2. 獨立盲測評估：使用未參與訓練訓練的測試集進行驗證，產出精確度、召回率、mAP 指標

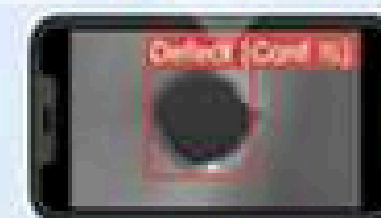


[階段四：實機手機拍攝與模型落地測試 (真實場景驗證)]

1. 實機硬體取像：使用 iPhone 13 + 防震機構 + 同軸環狀光源，擷取真實高反差影像

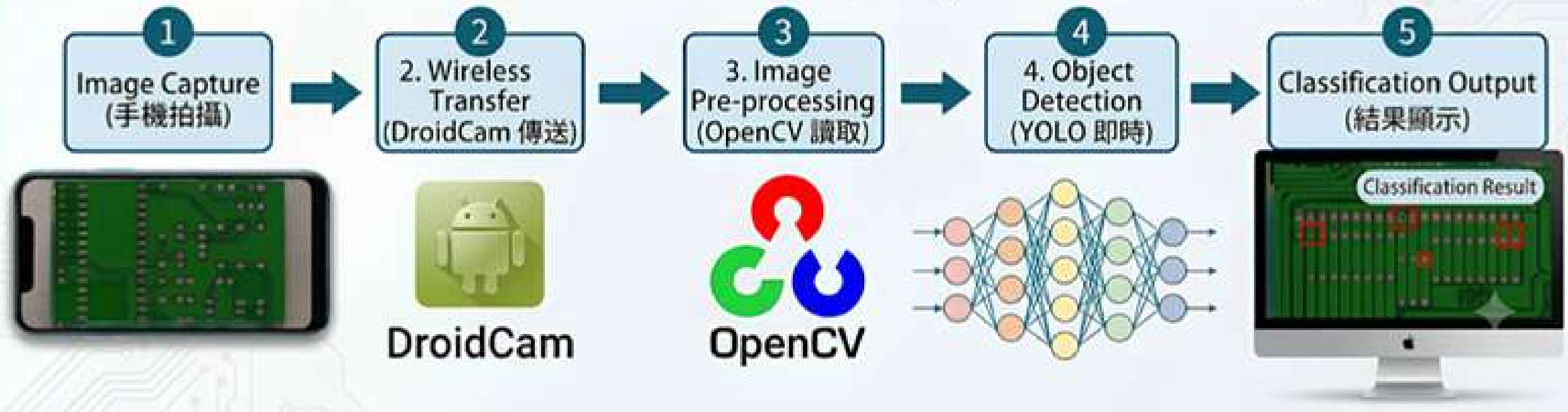


2. 即時預測視覺化：影像前處理 -> 模型推論 (信心閥值過濾) -> 精準標記缺孔邊界框



影像處理流程

PCB 孔洞缺失自動偵測系統流程圖 (Refined Workflow)



影像處理－格式化

核心演算法庫： 基於官方開源電腦視覺庫 OpenCV 開發。

```
# =====  
# 階段一：開源資料準備與格式化 (xml_to_txt.py & split_dataset.py)  
# =====  
def stage_1_data_pipeline(xml_dir, img_dir, output_dir):  
    """  
    1. 座標正規化：解析 Kaggle 原始 Pascal VOC (XML) 的絕對座標，數學轉換為 YOLO 相對比例。  
    2. 自動化架構：建立 images/train, labels/train 等雙軌目錄，防範資料外洩。  
    3. 科學化切分：將總資料集隨機洗牌，嚴格遵循 80% 訓練、10% 驗證、10% 測試的工業標準。  
    """  
    # 同時尋找 XML 內英文 'object' 與中文 '物件' 標籤  
    # 執行中心點 (x, y) 與寬高 (w, h) 的歸一化計算： $b\_center\_x = ((xmin + xmax) / 2.0) / img\_w$   
    pass
```


影像處理

```
# =====  
# 階段二：機器視覺前處理管線 (test.py & preprocess_dataset.py)  
# =====  
def stage_2_image_preprocessing(src_image_path):  
    """  
    核心前處理管線：從檢測目標影像輸入、機器視覺強化、到前處理特徵影像輸出。  
    """  
    # 1. 檢測目標影像輸入 (Image Input)  
    img_color = cv2.imread(src_image_path)  
  
    # 2. 影像灰階化 (Grayscale Conversion)  
    # 目的：將 RGB 彩色圖轉為單通道，剔除無關色彩資訊，降低運算維度以加速 AI 推論  
    img_gray = cv2.cvtColor(img_color, cv2.COLOR_BGR2GRAY)  
  
    # 3. 自適應直方圖均衡化 (CLAHE)  
    # 目的：限制對比度過度放大以抑制噪點，同時克服手機拍攝的局部過曝與陰影，銳化孔洞邊緣  
    clahe = cv2.createCLAHE(clipLimit=3.0, tileGridSize=(8, 8))  
    img_enhanced = clahe.apply(img_gray)  
  
    # 4. 高斯濾波降噪 (Gaussian Blur)  
    # 目的：使用 3x3 卷積核進行平滑濾波，消除手機或工業相機感測器帶來的高頻微小雜訊  
    img_blur = cv2.GaussianBlur(img_enhanced, (3, 3), 0)  
  
    # 5. 特徵影像輸出 (Image Output)  
    # 批次儲存為「前處理版專屬資料集」，確保 AI 訓練與實機測試具備絕對一致的光學基準  
    return img_blur
```


影像處理

```
# =====  
# 階段三：深度學習模型訓練與資料增強 (train_yolo.py)  
# =====  
def stage_3_train_model():  
    """  
    模型迭代訓練：使用輕量化 YOLOv8n 架構，輸入 640x640 尺寸影像。  
    線上資料增強 (Online Augmentation)：模型內建即時隨機翻轉、旋轉、HSV光影變換。  
    """  
    # 載入預訓練權重，準備進行遷移學習 (Transfer Learning)  
    model = YOLO('yolov8n.pt')  
  
    # 開始迭代訓練 50 個 Epochs，收斂邊界框損失 (Box Loss) 與類別損失 (Cls Loss)  
    model.train(  
        data='pcb_enhanced_data.yaml', # 對照組前處理設定檔  
        epochs=50, # 迭代次數  
        imgsz=640, # 網路輸入解像度  
        batch=8, # 批次批改量  
        device=0, # 啟用 GPU 運算加速  
        name='pcb_missing_hole_enhanced' # 儲存專案名稱  
    )  
    # 訓練完成自動保存最優權重檔案 `best.pt`
```


影像處理

```
# =====  
# 階段三：深度學習模型訓練與資料增強 (train_yolo.py)  
# =====  
def stage_3_train_model():  
    """  
    模型迭代訓練：使用輕量化 YOLOv8n 架構，輸入 640x640 尺寸影像。  
    線上資料增強 (Online Augmentation)：模型內建即時隨機翻轉、旋轉、HSV光影變換。  
    """  
    # 載入預訓練權重，準備進行遷移學習 (Transfer Learning)  
    model = YOLO('yolov8n.pt')  
  
    # 開始迭代訓練 50 個 Epochs，收斂邊界框損失 (Box Loss) 與類別損失 (Cls Loss)  
    model.train(  
        data='pcb_enhanced_data.yaml', # 對照組前處理設定檔  
        epochs=50, # 迭代次數  
        imgsz=640, # 網路輸入解像度  
        batch=8, # 批次批改量  
        device=0, # 啟用 GPU 運算加速  
        name='pcb_missing_hole_enhanced' # 儲存專案名稱  
    )  
    # 訓練完成自動保存最優權重檔案 `best.pt`
```

影像處理

```
# =====
# 階段四：量化盲測評估與實機視覺化推論 (evaluate_test.py & predict.py)
# =====
def stage_4_inference_and_validation():
    """
    1. 獨立盲測：使用從未參與訓練的 10% 獨立測試集，嚴格評估 Precision 與 Recall 指標。
    2. 落地測試：手機 (iPhone 13) 實際拍攝影像經前處理管線後，進行即時預測與視覺化標記。
    """
    # 載入前處理版最優權重模型
    model = YOLO(r'./runs/detect/pcb_missing_hole_enhanced/weights/best.pt')

    # 動作 A：終極盲測會考，嚴格把關工業檢測最重視的「漏檢率 (Recall)」
    metrics = model.val(data='pcb_enhanced_data.yaml', split='test')

    # 動作 B：真實產線場景推論 (套用手機實拍 PCB 局部特寫影像)
    # [技術亮點] 設定信心閾值 (conf=0.5) 過濾背景噪點，自動在原圖繪製 Bounding Box 標記缺孔位置
    results = model.predict(
        source=r'D:\VSCODE\PCB-missinghole\PCB_YOLO_Enhanced_Dataset\images\test',
        conf=0.5,          # 信心分數篩選門檻
        save=True,         # 自動存檔視覺化成果
        line_width=2       # 標記框線條粗細
    )

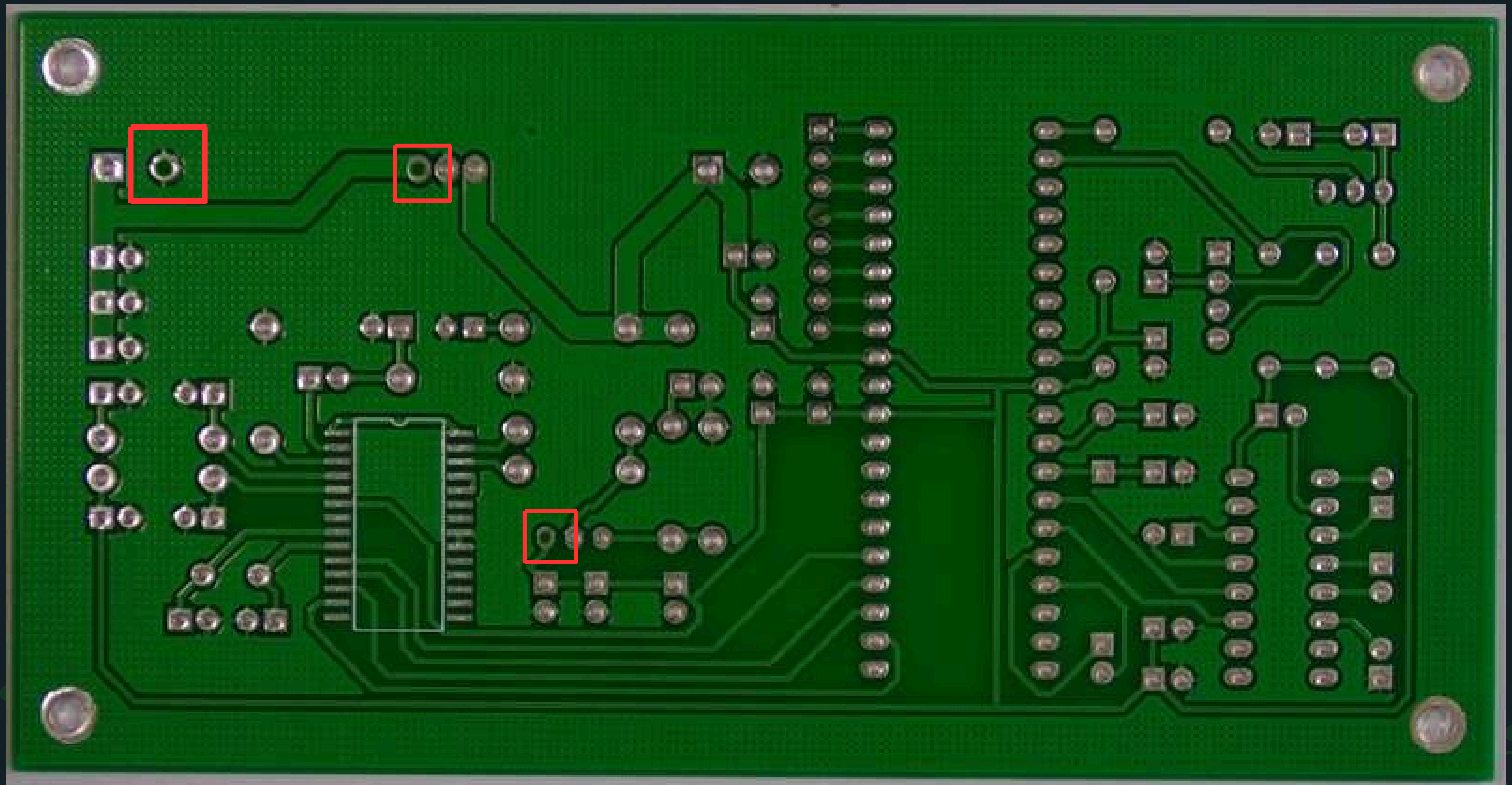
if __name__ == '__main__':
    print("🔗 印刷電路板 (PCB) 缺孔瑕疵檢測系統全流程運行中...")
    # 系統依循此端到端管線，實現工業級高檢出、低誤報之智慧 AOI 檢測
```


檢測結果指標

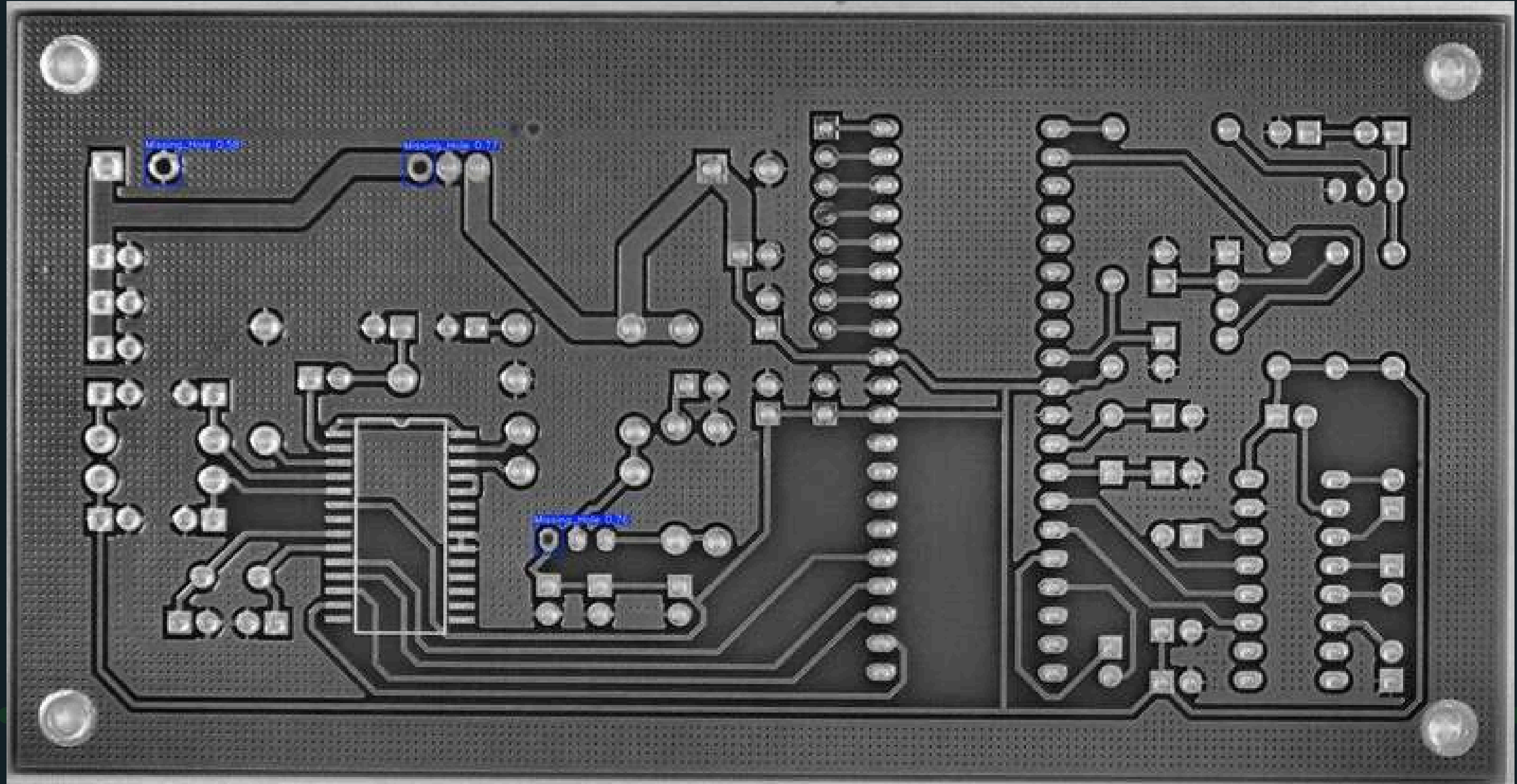
F1-score	Precise	Recall	mAP50	Inference Speed
0.95	1	0.913	0.961	14.3ms

- F1-score: 結合「精確率」與「召回率」的調和平均數，用於綜合評估模型在兩者之間的平衡度與整體效能穩定性。
- Precise: 在所有被模型判定為目標的結果中，真正屬於目標的比例。代表模型「防誤判」的能力。
- Recall: 在所有實際存在的目標中，被模型成功尋找出來的比例。代表模型「防漏判」的能力。
- mAP50: 當預測框與真實框的交集與聯集比 (IoU) 閾值設為 0.5 時，模型對所有類別精度的平均分數。是物件偵測領域中，衡量模型綜合分類與定位準確度的核心指標。
- Inference Speed: 模型對單張影像完成特徵提取到輸出預測結果所需的時間，用於衡量系統的硬體運算效率與即時性。

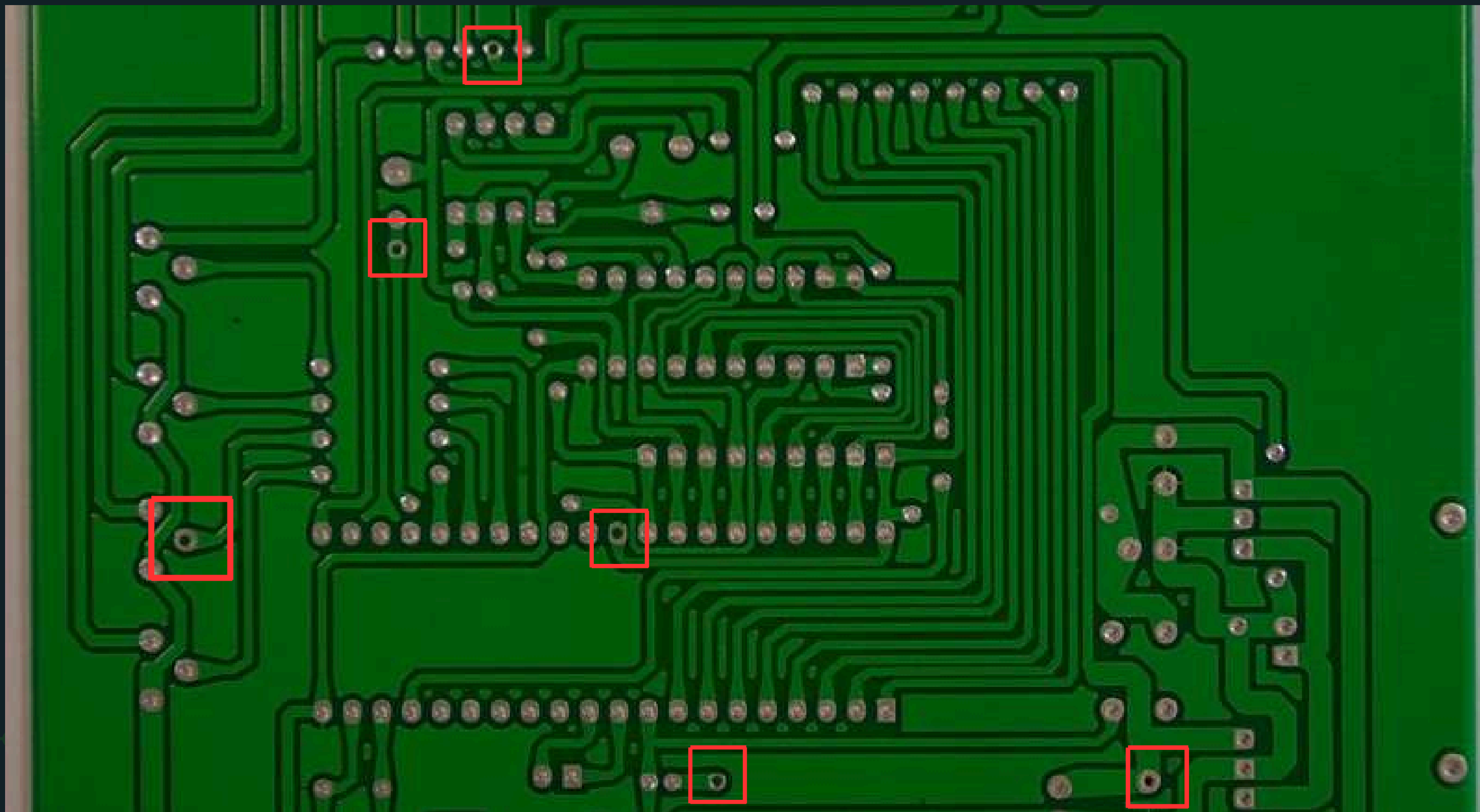
檢測結果1



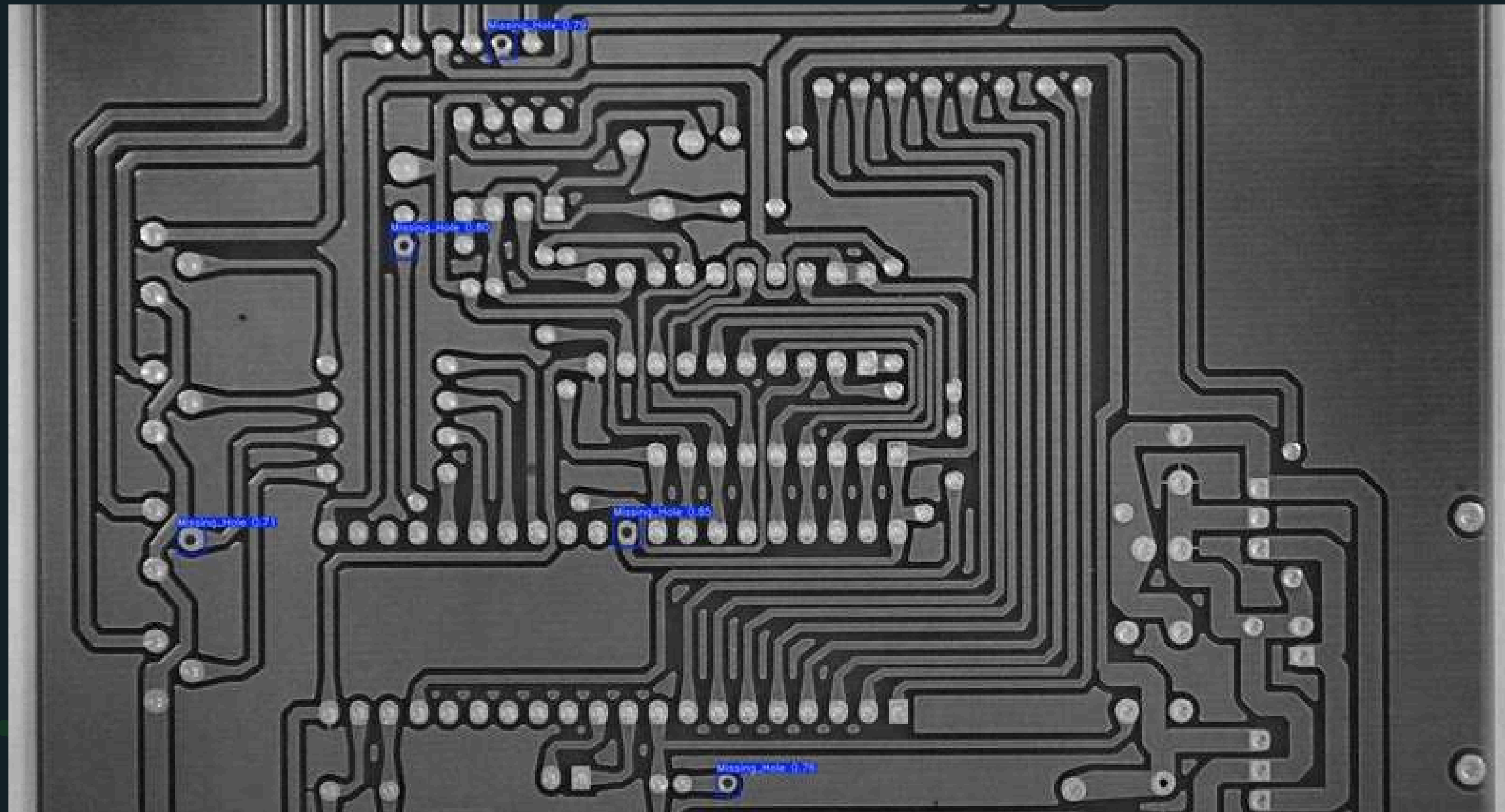
檢測結果1



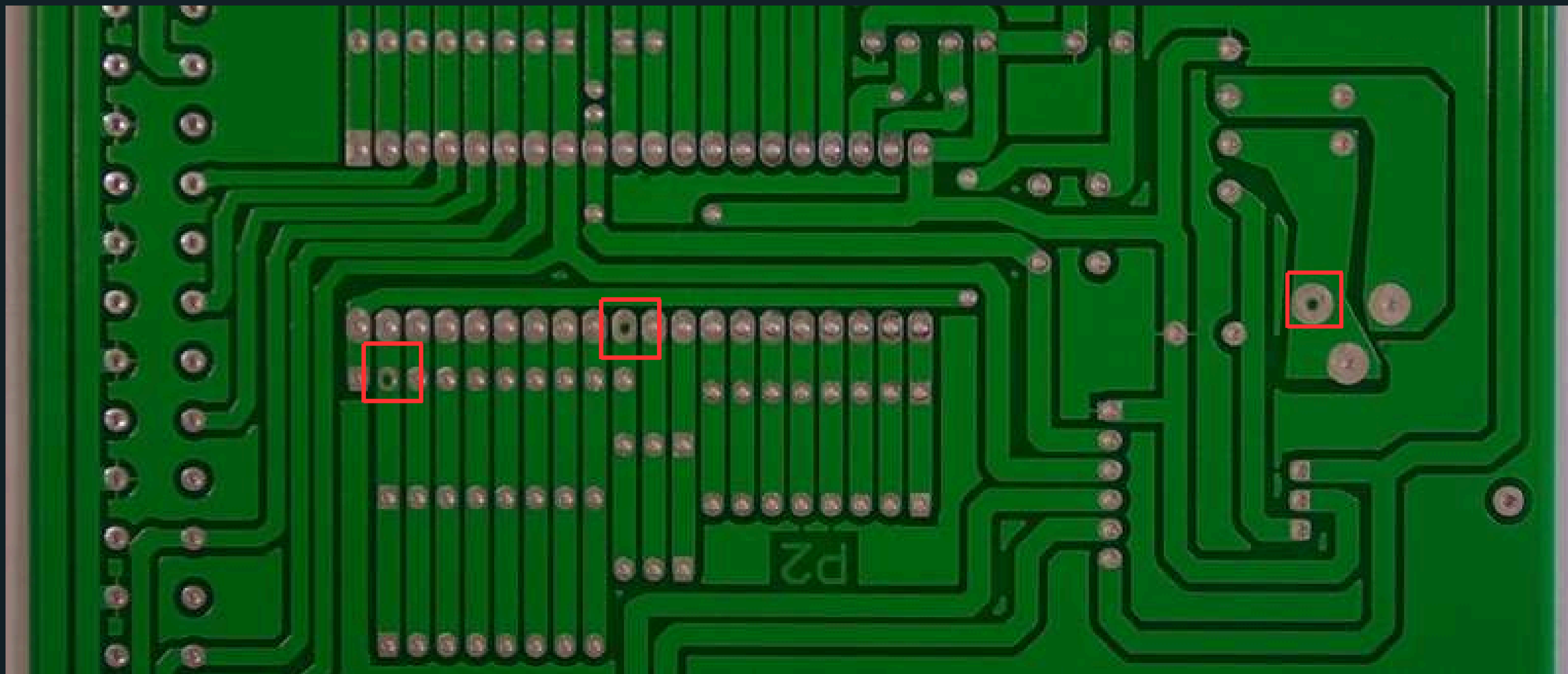
檢測結果2



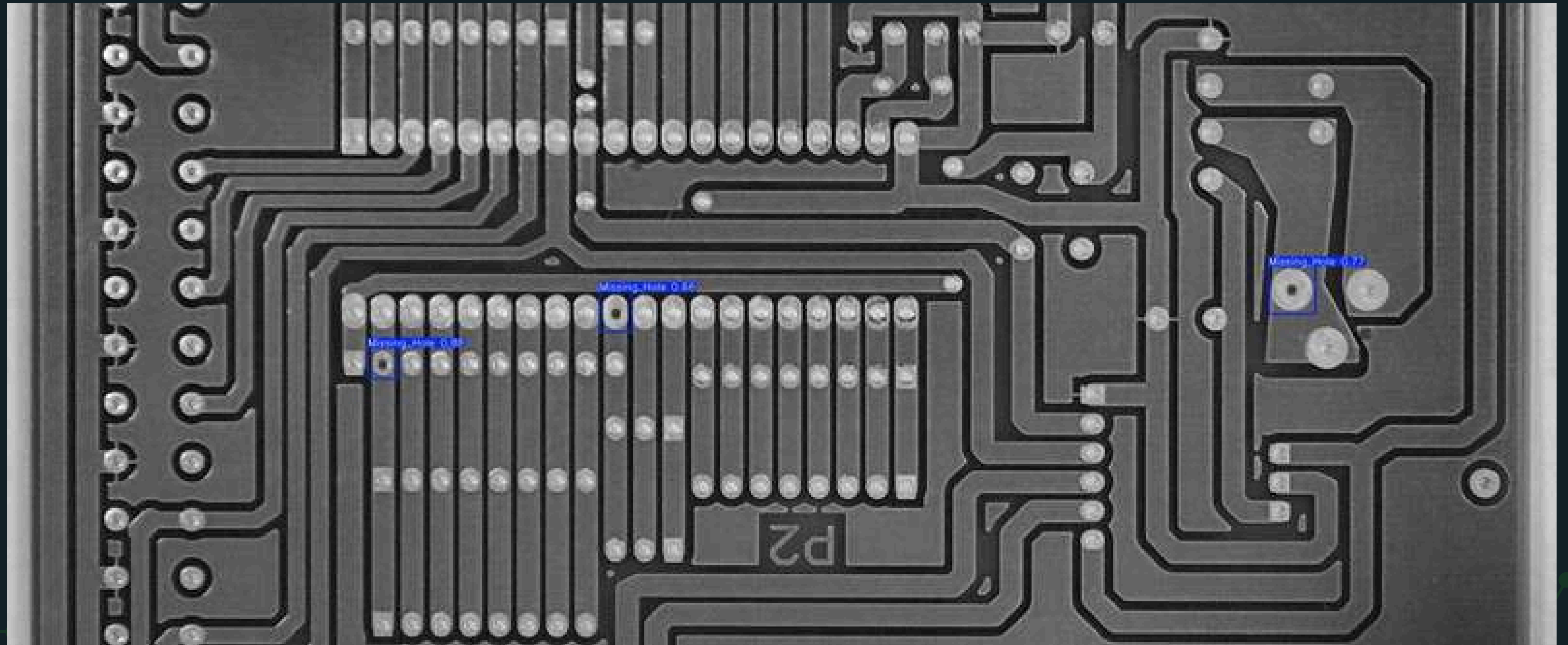
檢測結果2



檢測結果3

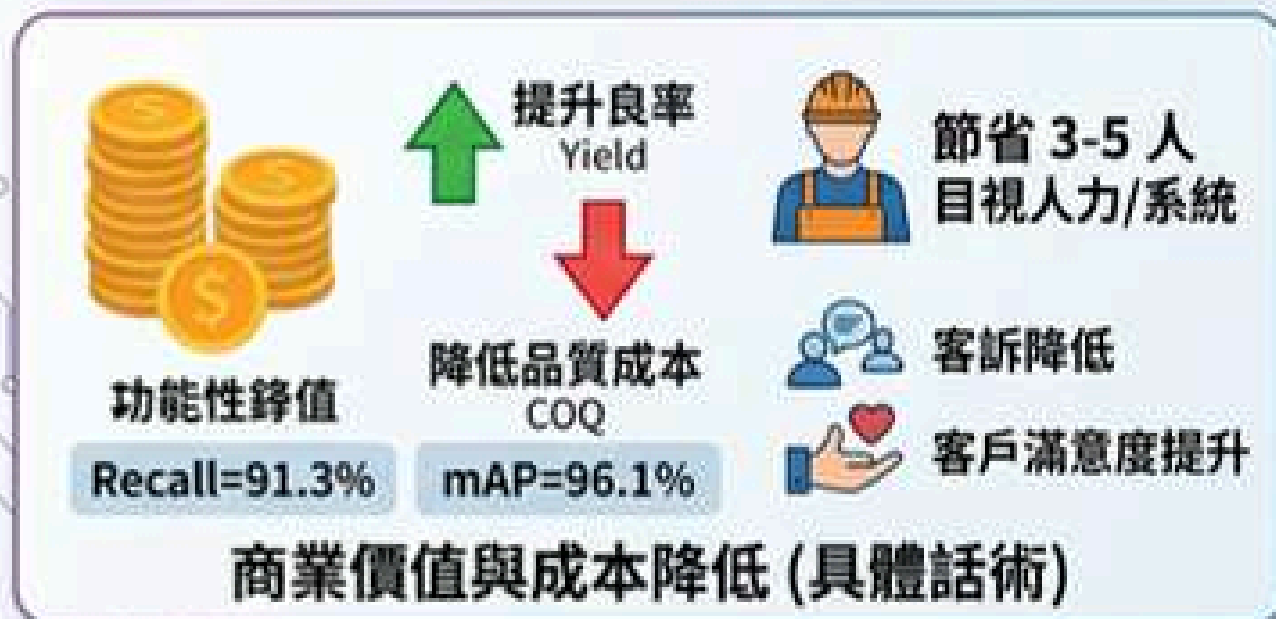
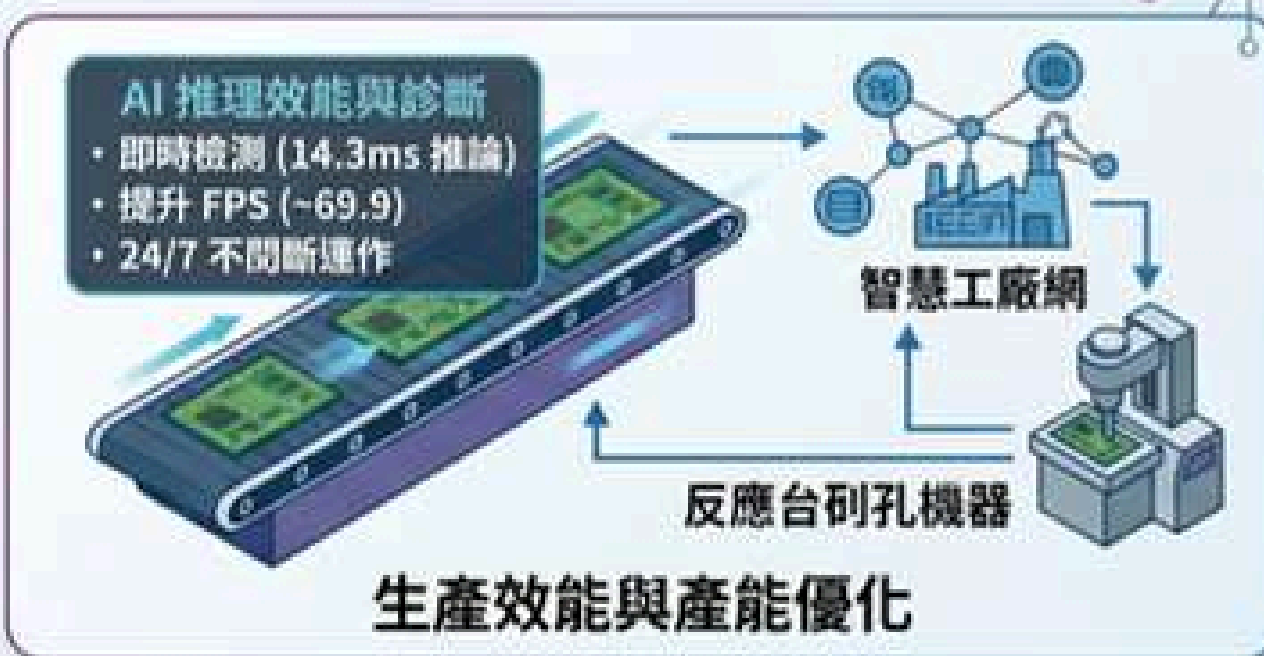
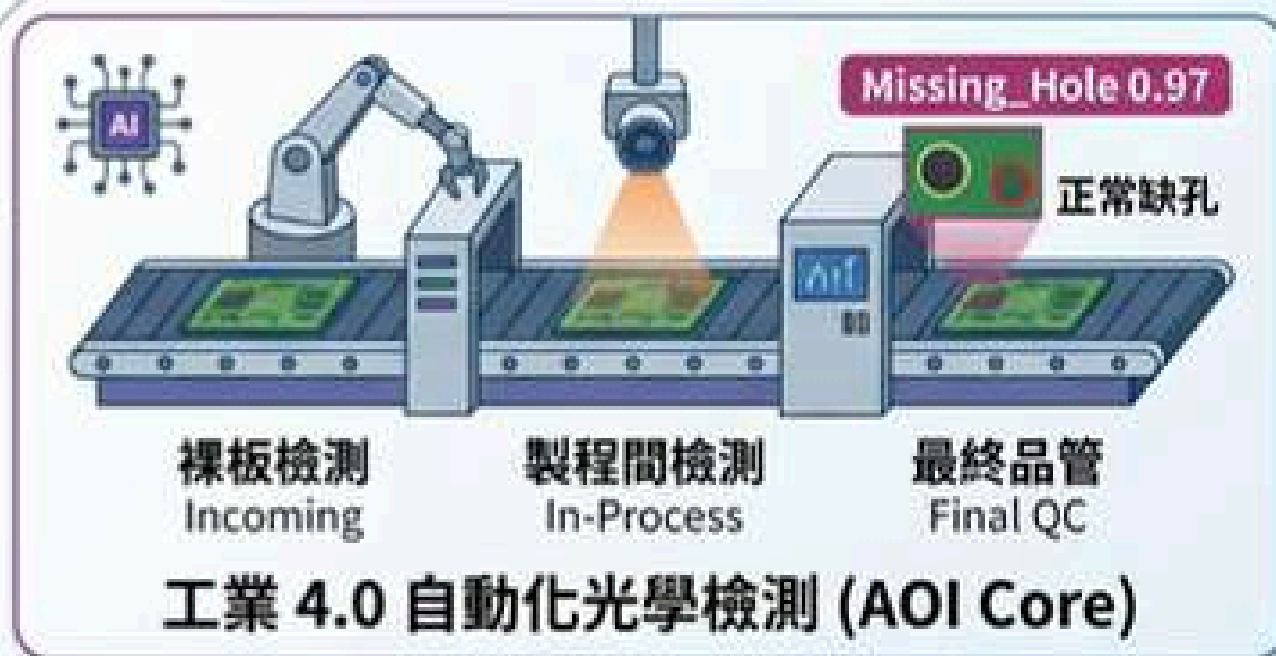


檢測結果3



專題應用

PCB 缺孔檢測專題應用場景總覽



與課程對應

課程知識點

專題實作應用說明

光學與照明設計

選用遠心鏡頭消除影像邊緣的透視變形，並搭配同軸光源有效抑制銅箔高反光干擾。

影像濾波與強化

實作灰階化降維，並套用高斯濾波 (Gaussian Blur) 消除工業相機感測器帶來的高頻雜訊。

特徵提取與幾何轉換

運用 ORB/SIFT 演算法提取特徵點，計算單應性矩陣完成 Template 與 Test 影像的幾何對齊。

形態學影像處理

利用開運算 (Opening) 的侵蝕與膨脹特性，精準抹除相減後產生的細碎輪廓殘影。

機器學習與分類

將處理後的 ROI 導入 YOLOv8 輕量化神經網路，進行特徵推論、分類判定與 Bounding Box 標記。

心得



製作心得與反思

這次專題讓我們深刻體會到，在深度學習模型訓練之前，前端的「光學環境建置」與傳統 CV 的「資料前處理」才是決定系統良率的關鍵。僅靠 AI 無法完美解決嚴重的反光干擾。唯有將硬體光學、傳統 OpenCV 影像處理與現代輕量化 YOLO 模型串聯，才能架構出真正符合產線高檢出、低誤報實務需求的自動化檢測系統。



工作分配與貢獻

- 蕭宇辰 (50%)：
負責硬體光學架構設計、OpenCV 8 大影像處理流程（特徵對齊、形態學處理）程式碼撰寫與演算法調校、簡報架構規劃。
- 羅詠倫 (50%)：
負責 DeepPCB 資料集清洗與標註格式轉換、YOLOv8 模型擴增與訓練、AI 模型與傳統演算法之整合測試與效能指標分析。



Thank You